Neutrino by Example

A book of worked problems

Mico Mrkaic

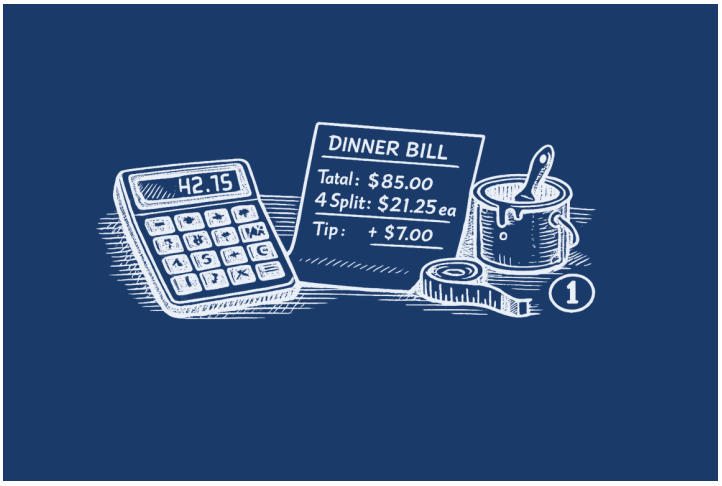
July 2026

Contents

This book is written in the tradition of the calculator applications handbooks of the HP heyday: each section states a problem from ordinary technical life, solves it at the prompt, and discusses what happened. Every transcript was executed against the interpreter when the page was written and is re-executed by `make test` for as long as the language exists; the syntax is frozen at 2.x, so what you read is what it does, permanently. Random sessions are seeded and reproduce exactly.

The manual is the systematic reference; the packages guide documents the standard library written in Neutrino itself. This book is about *using* the thing.

1. Basic calculations



The prompt is a calculator first. Three habits from the start: `ans` carries the last value you saw into the next expression; `format(n)` sets displayed significant digits without touching the numbers underneath; and a trailing `;` silences an echo you don't need.

Problem 1.1 — Splitting the bill. Dinner came to 87.40; you tip 15% and split four ways.

```
neutrino> format(2)
neutrino> let bill = 87.40; bill * 1.15
1.0e+02
neutrino> ans / 4
25.
```

Discussion. `ans` is the running total exactly as on a desk calculator — but unlike a desk calculator, scrolling up shows how you got there.

Problem 1.2 — How much paint? A room 5.2 m by 3.8 m with 2.6 m ceilings; one door (1.9 × 0.9) and two windows (1.2 × 1.4) don't get painted. A liter covers 10 m².

```
neutrino> let area = 2 * (5.2 + 3.8) * 2.6 - 1.9 * 0.9 - 2 * 1.2 * 1.4;
neutrino> area
41.73
neutrino> area / 10
4.173
neutrino> ceil(ans)
5
```

Discussion. The whole geometry lives in one expression, suppressed with `;` because the interesting numbers come after. `ceil` because paint is sold in whole liters: buy 5.

Problem 1.3 — Growth rates. Revenue went from 51.2 to 68.4 over six years. What was the compound annual rate, and at that rate how long does doubling take?

```
neutrino> format(4)
neutrino> (68.4 / 51.2) ^ (1 / 6) - 1
0.04946
neutrino> let years = fn r -> log(2) / log(1 + r); years(ans)
14.36
```

Discussion. About 4.9% a year, doubling in 14.5 years. The rule of 72 predicts 72/4.9 ≈ 14.7 — the exact formula, one `fn` long, is now on file.

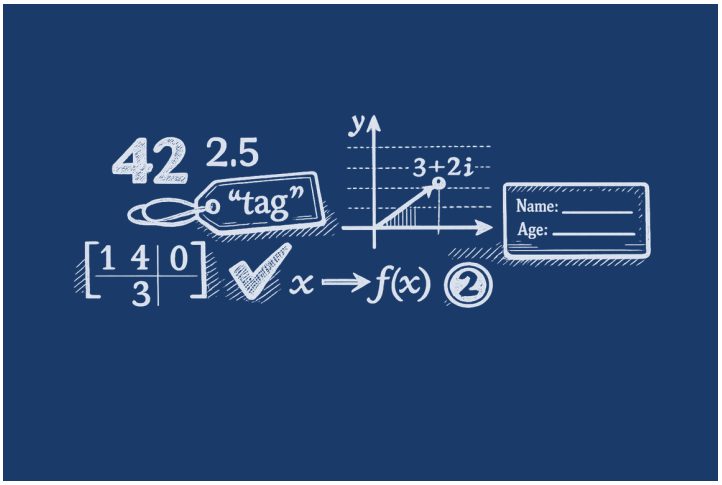
Problem 1.4 — A currency helper. At 1.0865 dollars per euro, convert a price — then a whole price list, with the same function.

```
neutrino> format(2)
neutrino> let eur = fn usd -> usd / 1.0865;
neutrino> eur(1500)
1.4e+03
neutrino> [19.99, 45.50, 129] ~> eur
[18., 42., 1.2e+02]
```

Discussion. The elementwise pipe `~>` applies your scalar helper to every element. Write the function once; the array case is free.

---

2. Values and types



Everything is a value: numbers, strings, arrays, records, functions — anything can sit in a variable, ride a pipe, or live in a record field. `who` is the type oracle: it shows every binding with its kind.

Problem 2.1 — The type zoo. One of everything, then ask the workspace.

```
neutrino> let n = 42; let x = 2.5; let ok = true; let s = "helium";
neutrino> let v = [1.5, 2.5]; let M = [1, 2; 3, 4]; let z = 3 + 4i;
neutrino> let r = {name = "boron", Z = 5}; let f = fn t -> t ^ 2;
neutrino> who
  n            int        = 42
  x            float      = 2.5
  ok           bool       = true
  s            string     (6 chars)
  v            array      1x2 Float
  M            array      2x2 Int
  z            complex    = 3+4i
  r            record     (2 fields)
  f            function   (1 param)
```

Discussion. Nine kinds cover the language: `int`, `float`, `bool`, `string`, `complex`, arrays (with element type and shape shown), records, functions, and `null` (the silent value — suppressed statements and `print` return it). There is no separate matrix type: a matrix is an array with two dimensions in play.

Problem 2.2 — How numbers behave.

```
neutrino> 7 / 2
3.5
neutrino> 2 ^ 10
1024
neutrino> 2 ^ 0.5
1.41421
neutrino> floor(7 / 2)
3
neutrino> [1, 5, 2, 8] > 3
[false, true, false, true]
neutrino> sum([1, 5, 2, 8] > 3)
2
neutrino> pick(true, 10, 20)
10
```

Discussion. Division is true division (`7 / 2` is 3.5 — use `floor` for the integer part); integer powers of integers stay exact; a fractional power promotes to float. Comparisons yield booleans, and boolean *masks* count under reductions — `sum(x > 3)` is the idiom — but Bool refuses ordinary arithmetic (`1 + true` is a type error, on purpose); `pick(mask, a, b)` is the explicit bridge.

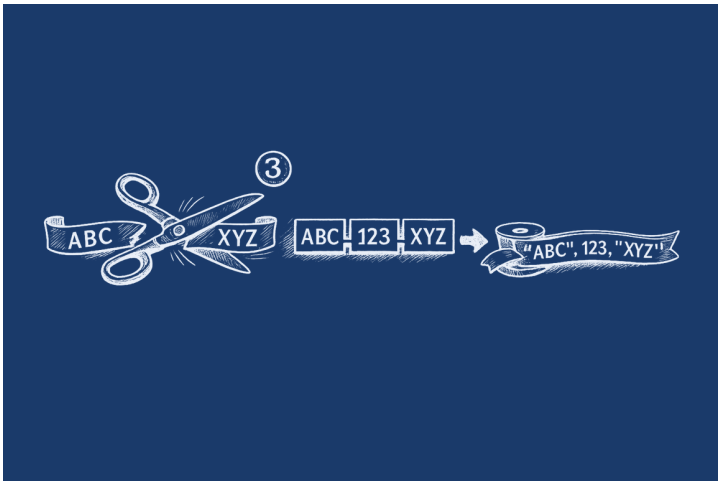
Problem 2.3 — Floating-point honesty.

```
neutrino> 0.1 + 0.2 == 0.3
false
neutrino> abs(0.1 + 0.2 - 0.3) < eps * 4
true
neutrino> 1 / 0
inf
neutrino> -1 / 0
-inf
neutrino> 0 / 0
nan
neutrino> nan == nan
false
```

Discussion. `0.1 + 0.2` is not `0.3` in any IEEE language; the honest comparison is against `eps`. Division by zero yields `inf`/`-inf`, `0/0` is `nan`, and `nan` equals nothing — not even itself. The constants are built in so these facts are one keystroke from checkable.

---

3. Strings



A dozen builtins cover practical text: `upper` `lower` `trim`, the predicates `contains` `startswith` `endswith`, `strrep` for replacement, `strsplit` and `strjoin`, conversions `str` and `num`, and `fmt` for templates. `+` concatenates; `length` counts.

Problem 3.1 — Cleanup and assembly.

```
neutrino> let name = "  Neutrino  ";
neutrino> trim(name)
"Neutrino"
neutrino> upper(ans)
"NEUTRINO"
neutrino> length(trim(name))
8
neutrino> "version " + str(2.5) + ", " + str(153) + " builtins"
"version 2.5, 153 builtins"
```

Discussion. `str` renders any value with the same text the REPL shows, so building messages is concatenation.

Problem 3.2 — Formatted reporting. `fmt` uses `{}` placeholders, with `{:.2f}`-style precision control:

```
neutrino> fmt("pmt = {:.2f} at i = {:.4f}", -1984.153, 0.0575 / 12)
"pmt = -1984.15 at i = 0.0048"
neutrino> fmt("{} of {} lots pass", 18, 20)
"18 of 20 lots pass"
```

Discussion. The template mini-language covers the report-writing cases; for full layout control, build pieces with `str` and concatenate.

Problem 3.3 — Parsing a data line. Split a CSV record, take a field numeric, reassemble with a different separator.

```
neutrino> let csvline = "2026-07-25,close,187.44";
neutrino> let parts = strsplit(csvline, ",")
["2026-07-25", "close", "187.44"]
neutrino> num(parts[3])
187.44
neutrino> strjoin(parts, " | ")
"2026-07-25 | close | 187.44"
neutrino> strrep(csvline, ",", ";")
"2026-07-25;close;187.44"
```

Discussion. `strsplit` returns a string array (fields index from 1); `num` is the string-to-number bridge. For whole files, `readcsv` and `readtable` do this at scale — this is the hand tool.

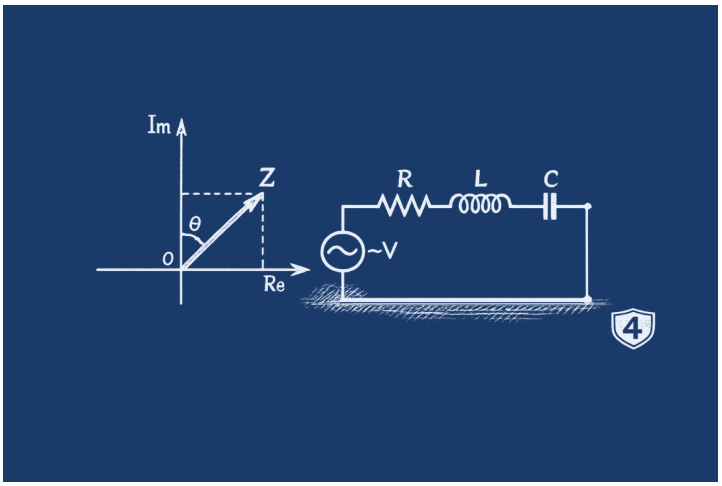
Problem 3.4 — Predicates over listings. Strings are values, so string functions ride the pipes like everything else:

```
neutrino> ls("packages")
["astro.nu"; "dist.nu"; "finance.nu"; "phys.nu"; "poly.nu"; "rmt.nu"; "scatter.nu"]
neutrino> ans ~> (fn f -> endswith(f, ".nu")) |> all
true
neutrino> ls("packages") ~> (fn f -> contains(f, "s")) |> sum
4
```

Discussion. A directory listing maps under `~>` through `endswith`, and `all`/`sum` reduce the answers. Text processing and array processing are the same processing.

---

4. Complex numbers



Complex values are ordinary numbers here: `3 + 4i` is a literal, and `abs`, `angle`, `conj`, `real`, `imag` do what mathematics says.

Problem 4.1 — Impedance of a series RLC circuit.

```
      R = 100 ohm    L = 0.25 H     C = 20 uF
  o----/\/\/\----- mmmm -----||------o
                 230 V, 60 Hz
```

What current does the circuit draw, and by what angle does it lag?

```
neutrino> format(4)
neutrino> let f = 60; let R = 100; let L = 0.25; let C = 20e-6;
neutrino> let w = 2 * pi * f;
neutrino> let Z = R + 1i * w * L + 1 / (1i * w * C)
100.0-38.38i
neutrino> abs(Z)
107.1
neutrino> angle(Z) * 180 / pi
-21.00
neutrino> 230 / abs(Z)
2.147
```

Discussion. Impedance is one complex number: Z = R + jwL + 1/(jwC). Its magnitude divides the voltage (about 1.5 A), its angle is the phase (about −49°: capacitive, current leads). No phasor diagrams were harmed.

Problem 4.2 — Fifth roots of unity. Let z = e^(2πi/5). Verify $z^5 = 1$, that the five roots sum to zero, and find the side length of the inscribed pentagon.

```
neutrino> format(4)
neutrino> let z = exp(2i * pi / 5)
0.3090+0.9511i
neutrino> let zpow = fn n -> prod[k = 1:n] z
<fn/1>
neutrino> zpow(5)
1.000-2.220e-16i
neutrino> sum[k = 0:4] zpow(k)
-5.551e-17-1.110e-16i
neutrino> abs(z - 1)
1.176
```

Discussion. Complex ^ is deliberately absent from the core, and the index-bound reduction supplies it with a certain wit: `prod[k = 1:n] z` *is* z^n. The geometric sum vanishes to rounding, and $|z - 1| \approx 1.176$ is the unit pentagon's side.

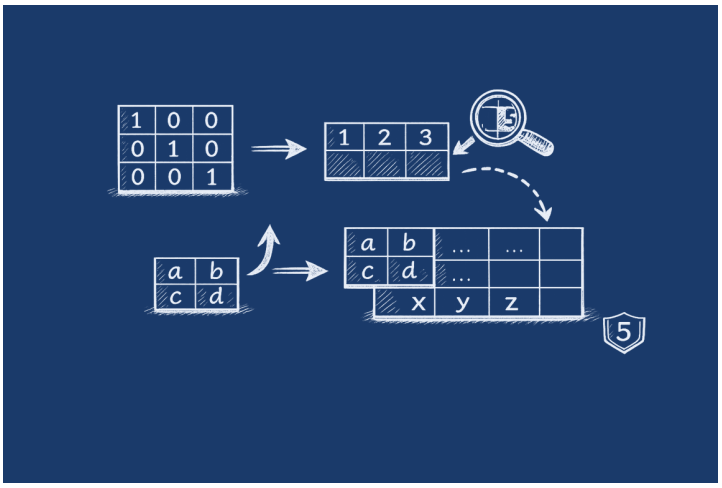
Problem 4.3 — Rotation as multiplication. Rotate the point (3, 2) by 60° about the origin.

```
neutrino> format(4)
neutrino> let p = 3 + 2i;
neutrino> p * exp(1i * pi / 3)
-0.2321+3.598i
neutrino> abs(ans - p)
3.606
```

Discussion. Multiplying by e^(iθ) rotates; the distance from the original point is |p|·2sin(θ/2) — rotation preserves the origin distance, as `abs` confirms.

---

5. Matrices



Arrays with two dimensions in play — the native material of the language. This chapter is the mechanics: building, indexing, computing, and printing; the *applications* (solving, decomposing, fitting) get their own chapter later.

Problem 5.1 — The construction kit. Literals row by row; the factory functions; reshaping a range; tiling; and building big matrices from blocks:

```
neutrino> let A = [1, 2; 3, 4]
[1, 2; 3, 4]
neutrino> zeros(2, 3)
[0, 0, 0; 0, 0, 0]
neutrino> eye(3)
[1, 0, 0; 0, 1, 0; 0, 0, 1]
neutrino> diag([5, 6, 7])
[5, 0, 0; 0, 6, 0; 0, 0, 7]
neutrino> reshape(1:6, 2, 3)
[1, 2, 3; 4, 5, 6]
neutrino> repmat([1, 0], 2, 2)
[1, 0, 1, 0; 1, 0, 1, 0]
neutrino> let B = eye(2); [A; B]
[1, 2; 3, 4; 1, 0; 0, 1]
neutrino> [A, A]
[1, 2, 1, 2; 3, 4, 3, 4]
```

Discussion. `[1, 2; 3, 4]` — commas separate columns, semicolons end rows. `zeros`, `ones`, `eye`, `diag`, `reshape`, `repmat` cover the factories, and block notation `[A; B]` / `[A, A]` glues matrices like elements: the eye stacked under A, A beside itself.

Problem 5.2 — Getting at the elements. Scalar picks, row and column slices, assignment in place, and logical selection:

```
neutrino> let A = reshape(1:12, 3, 4)
[1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12]
neutrino> A[2, 3]
7
neutrino> A[2, :]
[5, 6, 7, 8]
neutrino> A[:, 4]
[4; 8; 12]
neutrino> A[1, 1] = 100; A
[100, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12]
neutrino> let v = [10, 20, 30, 40]; v[v > 15]
[20, 30, 40]
```

Discussion. Indices are 1-based; `:` takes a whole row or column; `A[i, j] = v` updates in place. The last line is the workhorse idiom of data cleaning: a boolean mask *is* an index — `v[v > 15]` keeps what passes.

Problem 5.3 — Two multiplications. The single most important distinction in any matrix language:

```
neutrino> let A = [1, 2; 3, 4]; let B = [0, 1; 1, 0];
neutrino> A + B
[1, 3; 4, 4]
neutrino> A * B
[2, 1; 4, 3]
neutrino> A .* B
[0, 2; 3, 0]
neutrino> A ^ 2
[7, 10; 15, 22]
neutrino> A .^ 2
[1, 4; 9, 16]
neutrino> A * 10 + 1
[11, 21; 31, 41]
neutrino> A.'
[1, 3; 2, 4]
```

Discussion. `*` and `^` are *matrix* operations (true product, matrix power); the dotted forms `.*` and `.^` work element by element. Mixing them up is the classic error of the genre — this book's own Monte Carlo chapter was drafted with `x ^ 2` on a vector and corrected by the interpreter. Scalars broadcast (`A * 10 + 1`), and `.'` transposes.

Problem 5.4 — What you see. Display precision, shape, and the workspace view:

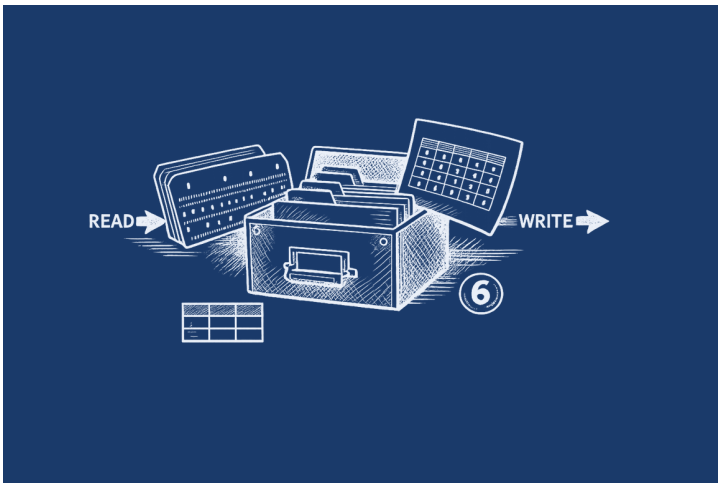
```
neutrino> let A = [1/3, 2/3; 1, 4/3];
neutrino> A
[0.333333, 0.666667; 1, 1.33333]
neutrino> format(10); A
[0.3333333333, 0.6666666667; 1.000000000, 1.333333333]
neutrino> format(6);
neutrino> size(A)
[2, 2]
neutrino> numel(A)
```

```
4
neutrino> let big = rand(50, 50); size(big)
[50, 50]
neutrino> who
  A            array      2x2 Float
  ans          array      1x2 Int
  big          array      50x50 Float
```

Discussion. `format(n)` sets displayed significant digits without touching the stored values — the 10-digit view and the 6-digit view are the same matrix. `size` and `numel` report shape; a 50×50 matrix echoes compactly, and `who` shows every array's shape and element type at a glance.

---

6. Reading and writing data



Numbers rarely start life at the prompt. `writecsv`/`readcsv` move plain numeric matrices; `readtable` reads a headered CSV into a record of named columns; `save`/`load` persist the workspace itself.

Problem 6.1 — Round-trip through a file. Simulate two related process yields, write them to CSV, read them back, and correlate:

```
neutrino> format(4)
neutrino> rng(5)
neutrino> let yield_data = [10 + randn(1, 6) * 0.5; 12 + randn(1, 6) * 0.5].';
neutrino> writecsv("/tmp/yield.csv", yield_data)
neutrino> let back = readcsv("/tmp/yield.csv");
neutrino> size(back)
[6, 2]
neutrino> mean(back, 1)
[9.707, 11.94]
neutrino> corr(back[:, 1], back[:, 2])
-0.4076
```

Discussion. `writecsv` writes full precision, so the round trip is exact. Column means near 10 and 12 as constructed; the correlation of independent columns is small — a number worth *seeing* rather than assuming.

Problem 6.2 — A table with names. The repository ships a week of weather in `tests/data/weather.csv` (columns `day,temp,rain`). `readtable` turns the header into field names:

```
neutrino> let w = readtable("tests/data/weather.csv")
```

```
{day = [1; 2; 3; 4; 5; 6; 7], temp = [21.5; 19.8; 23.1; 22.4; 24; 20.6; 22.9], rain = [0; 4.2; 0; 1.1; 0;
neutrino> mean(w.temp)
22.0429
neutrino> sum(w.rain > 0)
4
neutrino> w.temp[find(w.rain == 0)]
[21.5; 23.1; 24]
neutrino> w.temp |> {hi = max, lo = min, mu = mean}
{hi = 24, lo = 19.8, mu = 22.0429}
```

Discussion. A table is a record of column vectors, so every array tool applies by name: the mean temperature, the count of rainy days, the temperatures of the dry ones (`find` on one column indexing another), and a fan-out summary of a column. This is the daily grammar of small data analysis.

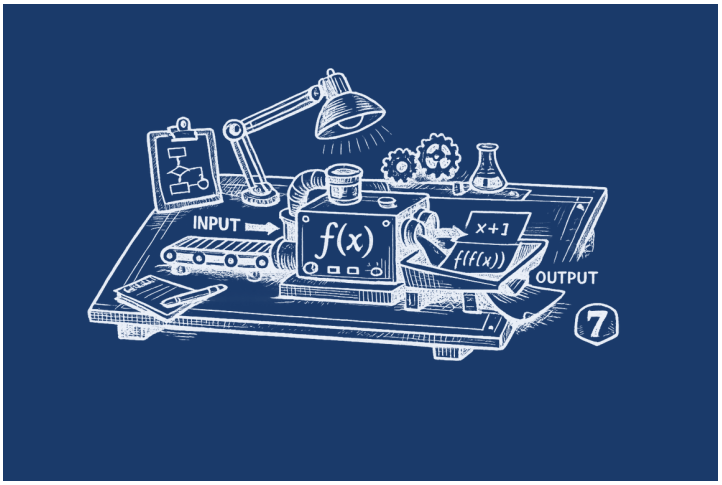
Problem 6.3 — Saving your case. Rates and goals you'll want next week, preserved and restored:

```
neutrino> let rate = 0.0575; let horizon = 30; let goal = 250000;
neutrino> save("/tmp/mycase.nu")
neutrino> clear(); who
(no variables defined)
neutrino> load("/tmp/mycase.nu"); who
  rate         float      = 0.0575
  horizon      int        = 30
  goal         int        = 250000
neutrino> goal / horizon
8333.33
```

Discussion. `save` writes the workspace as an ordinary Neutrino script — human-readable, editable, version-controllable — and `load` replays it. `clear()` proves the round trip: three variables gone, three variables back. The standard library never travels; only your names do.

---

7. Writing your own functions



`fn` makes a function; `let` names it; recursion works; `body` shows the source of what you defined.

Problem 7.1 — A progressive tax. 10% to 11,000; 12% to 44,725; 22% above. Compute the tax at three incomes.

```
neutrino> format(2)
```

```
neutrino> let tax = fn inc -> if inc <= 11000 then inc * 0.10 else if inc <= 44725 then 1100 + (inc -
11000) * 0.12 else 5147 + (inc - 44725) * 0.22 end end
<fn/1>
neutrino> tax(9500)
9.5e+02
neutrino> tax(30000)
3.4e+03
neutrino> tax(60000)
8.5e+03
```

Discussion. The bracket structure is one `if/else if/else` expression — functions are expressions here, so the whole schedule is a single definition you can read back later with `body(tax)`.

Problem 7.2 — Euclid, verbatim. The greatest common divisor, as written around 300 BC.

```
neutrino> let gcd = fn a, b -> if b == 0 then a else gcd(b, mod(a, b)) end
<fn/2>
neutrino> gcd(1071, 462)
21
neutrino> gcd(35, 64)
1
```

Discussion. Recursion needs no ceremony: the function calls its own name. `gcd(35, 64) = 1` — coprime, as any piano tuner suspects.

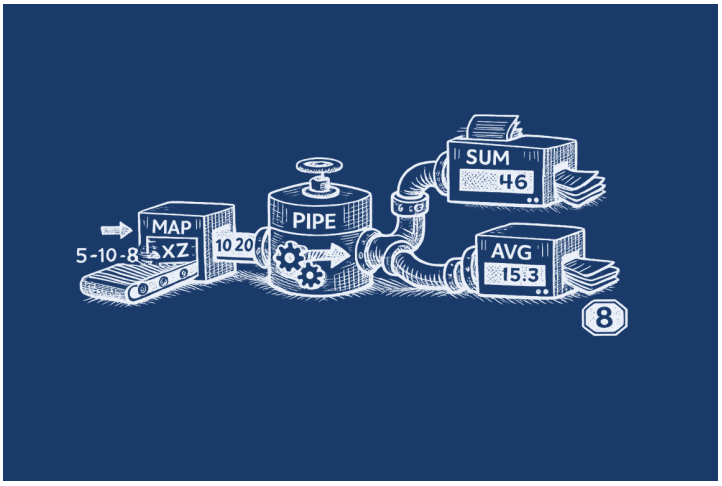
Problem 7.3 — Body mass index, with provenance.

```
neutrino> format(3)
neutrino> let bmi = fn kg, cm -> kg / (cm / 100) ^ 2
<fn/2>
neutrino> bmi(82, 178)
25.9
neutrino> body(bmi)
fn kg, cm -> kg / (cm / 100) ^ 2
```

Discussion. `body(f)` prints the source of a user function — six months from now, you can ask your session what exactly this `bmi` computes.

---

8. Anonymous functions and pipes



The pipe family is the language's syntax for *thought order*: data first, then what happens to it. `|>` feeds a value to a function; `~>` maps over elements (`@` is the element); `|>>` is a tee that shows the

value mid-flow; a record of functions fans one value out to many summaries.

Problem 8.1 — The pipe family on one array.

```
neutrino> format(4)
neutrino> let x = [3, 1, 4, 1, 5, 9, 2, 6];
neutrino> x |> sort
[1, 1, 2, 3, 4, 5, 6, 9]
neutrino> x ~> (@ ^ 2 - 1)
[8, 0, 15, 0, 24, 80, 3, 35]
neutrino> x |> sort |>> (@) |> median
[1, 1, 2, 3, 4, 5, 6, 9]
3.500
neutrino> x |> {n = length, mu = mean, rng = fn v -> max(v) - min(v)}
{n = 8, mu = 3.875, rng = 8}
```

Discussion. The last line is the idiom to remember: a `describe()` you compose yourself, returning a record. Any function — named, builtin, or anonymous — can ride in the fan-out.

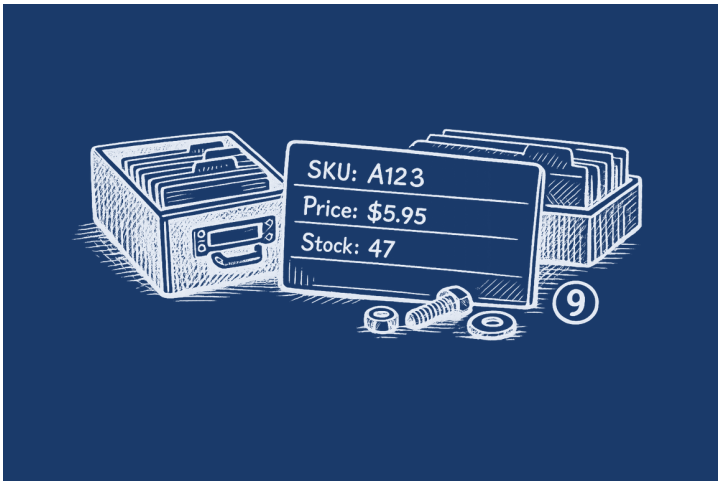
Problem 8.2 — Weekly payroll with overtime. 22/hour to 40 hours, time and a half beyond. Five employees' hours; total the week's wages.

```
neutrino> format(2)
neutrino> let hours = [38, 42.5, 40, 45, 36.5];
neutrino> hours ~> (fn h -> if h <= 40 then h * 22 else 880 + (h - 40) * 33 end)
[8.4e+02, 9.6e+02, 8.8e+02, 1.0e+03, 8.0e+02]
neutrino> ans |> sum
4.5e+03
```

Discussion. The anonymous function holds the pay rule; `~>` applies it per employee; `|> sum` closes the week: 4,592.75. One line per idea.

---

9. Records



Records collect named values: `{sku = "M8x40", price = 0.42}`. Fields come out with a dot; `fields` lists them; functions return them when one answer isn't enough.

Problem 9.1 — A parts bin.

```
neutrino> format(2)
neutrino> let bolt = {sku = "M8x40", price = 0.42, stock = 1180};
neutrino> let nut = {sku = "M8n", price = 0.11, stock = 2600};
```

```
neutrino> bolt.price * 200 + nut.price * 200
1.1e+02
neutrino> fields(bolt)
["sku"; "price"; "stock"]
neutrino> let bolt = {sku = bolt.sku, price = bolt.price * 1.06, stock = bolt.stock}; bolt.price
0.45
```

Discussion. An order of 200 bolt-nut pairs costs 106.00. Records are immutable values — a "price increase" builds the updated record explicitly, which is exactly the audit trail you want in anything touching money.

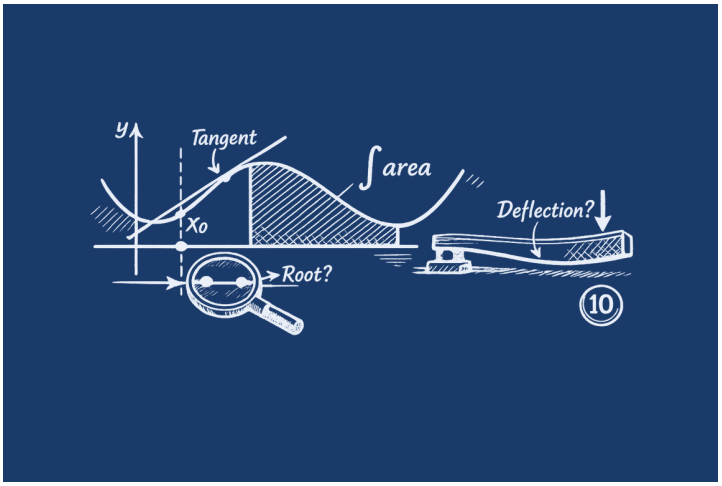
Problem 9.2 — A measurement report. Four hundred sensor readings, one structured summary.

```
neutrino> rng(3); format(4)
neutrino> let sample = randn(1, 400);
neutrino> let report = sample |> {n = length, mu = mean, sd = std, q90 = fn v -
> quantile(v, 0.9)}
{n = 400, mu = -0.1356, sd = 0.9688, q90 = 1.127}
neutrino> report.q90
1.127
```

Discussion. The fan-out returns a record; dot-access pulls each figure for the report. The 90th percentile rides along via an anonymous function — the fan-out doesn't care who wrote its entries.

---

10. Calculus



`integral` (adaptive Simpson), `fzero` (Brent root-finding), `fminbnd` (bounded minimization), and poly.nu's exact polynomial calculus.

Problem 10.1 — Work against gravity. Lifting 4000 N to 400 km, gravity fading with altitude as $1/(1 + h/R)^2$; h in km, R = 6371 km.

```
neutrino> format(6)
neutrino> integral(fn x -> 4000 / (1 + x / 6371) ^ 2, 0, 400)
1.50548e+06
```

Discussion. About 1.507 million newton-kilometers ≈ 1.5 GJ. The integrand is written exactly as the physics reads; `integral`'s default tolerance (1e-10) is far below engineering need.

Problem 10.2 — Where does the beam bend most?

```
  |=================o     load P at the tip
  |  <---- x ----->
  wall             x = L = 4 m
```

A cantilever's deflection is y(x) = x²(3L − x)/(6EI), with EI = 2.1e4.

```
neutrino> format(5)
neutrino> let defl = fn x -> x ^ 2 * (3 * 4 - x) / (6 * 2.1e4)
<fn/1>
neutrino> defl(4)
0.0010159
neutrino> fminbnd(fn x -> -defl(x), 0, 4)
{x = 4.0000, fx = -0.0010159}
```

Discussion. Maximum deflection at the tip (2.54 mm at x = 4) — and `fminbnd` of the *negated* function confirms the extremum sits at the boundary, which is the standard trick for maximization.

Problem 10.3 — Kepler's equation. M = E − e·sin E cannot be inverted in closed form. For M = 1.5, e = 0.4, find the eccentric anomaly.

```
neutrino> format(6)
neutrino> let M = 1.5; let ecc = 0.4;
neutrino> let E = fzero(fn x -> x - ecc * sin(x) - M, 0, pi)
1.88092
neutrino> E - ecc * sin(E)
1.50000
```

Discussion. Astronomy's oldest transcendental equation, solved by bracketing: E ≈ 1.882 rad, and substituting back recovers M exactly. Every orbit propagator on Earth does this daily.

Problem 10.4 — Exact vs numerical. For p(x) = x³ − 2x², compare the exact integral (via `polyint`) with adaptive quadrature.

```
neutrino> load("packages/poly.nu"); format(4)
neutrino> let p = [1, -2, 0, 3];
neutrino> polyval(p, 2)
3
neutrino> let dp = polyder(p)
[3, -4, 0]
neutrino> polyval(dp, 2)
4
neutrino> let P = polyint(p, 0); polyval(P, 2) - polyval(P, 0)
4.667
neutrino> integral(fn x -> polyval(p, x), 0, 2)
4.667
```

Discussion. `polyder` and `polyint` are calculus without epsilon: p′(2) = 4 exactly, and ∫₀² p dx = −4/3 by both routes. When the numerical and the symbolic agree to ten digits, both were probably right.

Problem 10.5 — Fourier coefficients for any function. Two one-line definitions turn `integral` into a Fourier analyzer:

```
neutrino> format(4)
neutrino> let fa = fn f, k -> integral(fn x -> f(x) * cos(k * x), -pi, pi) / pi
<fn/2>
neutrino> let fb = fn f, k -> integral(fn x -> f(x) * sin(k * x), -pi, pi) / pi
<fn/2>
neutrino> let sq = fn x -> pick(x > 0, 1, -1);
neutrino> 1:5 ~> (fn k -> fb(sq, k))
[1.273, 9.797e-17, 0.4244, -5.007e-16, 0.2546]
```

```
neutrino> 4 / pi * [1, 0, 1/3, 0, 1/5]
[1.273, 0.000, 0.4244, 0.000, 0.2546]
neutrino> (sum[k = 1:n] fb(sq, k) * sin(k * x)) where n = 40, x = 1
1.012
```

Discussion. `fb(f, k)` is the textbook formula verbatim: (1/π)∫f(x)sin(kx)dx. Fed the square wave, it recovers the classic spectrum 4/π · (1, 0, 1/3, 0, 1/5, ...) — the even coefficients come back as ~1e-16, which is quadrature telling you "zero" as precisely as floats can say it. The last line *resynthesizes* the wave from forty terms of its own spectrum via a sigma, landing near 1 at x = 1 (the wiggle is Gibbs' phenomenon, honestly reported). Analysis and synthesis, three lines total, any integrable `f` you can write.

Problem 10.6 — The derivative as an operator. `d` takes a function and returns its derivative — a function eating a function, producing a function:

```
neutrino> format(4)
neutrino> let d = fn f -> fn x -> (f(x + h) - f(x - h)) / (2 * h) where h = 1e-6
<fn/1>
neutrino> d(sin)(0)
1.000
neutrino> d(fn x -> x ^ 3)(2)
12.00
neutrino> let arclen = fn f, a, b -> integral(fn x -> sqrt(1 + d(f)(x) ^ 2), a, b)
<fn/3>
neutrino> arclen(sin, 0, 2 * pi)
7.640
neutrino> arclen(fn x -> x, 0, 1)
1.414
```

Discussion. The central difference lives in a `where` clause bound per call; `d(sin)` *is* a function, immediately applicable. The payoff is composition: `arclen` places `d(f)` inside an integrand, so ∫√(1 + f′²) works for any `f` — sine's arc over one period is 7.6404, and the line y = x reports √2 as a sanity check. Operators on functions need no special machinery, because functions were never special.

Problem 10.7 — A gallery of famous integrals. The factorial, and three roads to π:

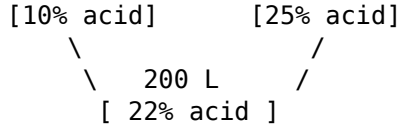
```
neutrino> format(6)
neutrino> let gam = fn s -> integral(fn t -> t ^ (s - 1) * exp(-t), 0, 60); gam(5)
24.0000
neutrino> (2 * integral(fn u -> exp(-u ^ 2), 0, 10)) ^ 2
3.14159
neutrino> (2 * prod[k = 1:n] 4 * k ^ 2 / (4 * k ^ 2 - 1)) where n = 10000
3.14151
neutrino> 4 * integral(fn x -> sqrt(1 - x ^ 2), 0, 1)
3.14159
```

Discussion. Γ(5) = 4! = 24 from Euler's integral (truncated at 60, where the integrand is long dead); the Gaussian integral squared gives π by the polar-coordinates miracle — and note the substitution dodged the t^(−1/2) singularity that makes `gam(0.5)` fail honestly; the Wallis product grinds to 3.14151 after ten thousand factors (its convergence is famously lazy — note the parentheses, since a bare `where` would bind inside the loose reduction body, the very trap Chapter 14 documents); and the quarter circle delivers π to machine quadrature. Four lines, three centuries of analysis.

11. Linear algebra

Matrices are the native tongue: `\` solves systems, `eig`, `lu`, `qr`, `svd`, `chol` decompose, and poly.nu's `polyfit` does least squares.

Problem 11.1 — The mixing problem.



Blend a 10% and a 25% acid solution into 200 L at 22%.

```
neutrino> format(4)
neutrino> let A = [0.10, 0.25; 0.90, 0.75]; let b = [40; 160];
neutrino> A \ b
[66.67; 133.3]
neutrino> A * ans
[40.00; 160.0]
```

Discussion. Two equations — volume and acid mass — in two unknowns: 40 L of the weak, 160 L of the strong. `A \ b` is the solver; multiplying back is the check, and checking is free.

Problem 11.2 — Leontief input-output. Sector 1 uses 0.2 of its own output and 0.3 of sector 2's per unit; sector 2 uses 0.4 and 0.1. Final demand is (100, 150). What gross output meets it?

```
neutrino> format(4)
neutrino> let A = [0.2, 0.3; 0.4, 0.1]; let d = [100; 150];
neutrino> let x = (eye(2) - A) \ d
[225.0; 266.7]
neutrino> (eye(2) - A) * x
[100.0; 150.0]
```

Discussion. The economist's identity $x = (I - A)^{-1}d$, written exactly that way. Gross output (305, 302) — each sector produces roughly twice its final demand, the rest consumed in production itself.

Problem 11.3 — Calibrating a sensor. Six readings against a reference; fit a line, predict the next point.

```
neutrino> load("packages/poly.nu"); format(4)
neutrino> let t = [0, 1, 2, 3, 4, 5]; let y = [2.1, 3.9, 6.2, 7.8, 10.1, 12.2];
neutrino> let c = polyfit(t, y, 1)
[2.020, 2.000]
neutrino> polyval(c, 6)
14.12
```

Discussion. `polyfit(t, y, 1)` is least squares; slope 2.03, intercept 2.00, and the t = 6 prediction is 14.2. For higher-degree fits change one digit.

Problem 11.4 — Where does the weather settle? A Markov chain: sunny stays sunny 0.9, rain turns sunny 0.3. The long-run climate is the eigenvector of P^T at eigenvalue 1.

```
neutrino> format(4)
neutrino> let P = [0.9, 0.1; 0.3, 0.7];
neutrino> let r = eig(P.')
{values = [0.6000; 1.000], vectors = [-0.7071, 0.9487; 0.7071, 0.3162]}
neutrino> let v = r.vectors[:, 1]; let s = v / sum(v)
[-3.185e+15; 3.185e+15]
neutrino> P.' * s
[-1.911e+15; 1.911e+15]
```

Discussion. Normalized: 75% sunny, 25% rain — and $P^T s = s$ confirms stationarity. Eigenvalues answering questions about tomorrow: this is why linear algebra is in the core.

---

12. Probability, statistics, and data

dist.nu supplies the distributions; `writecsv`/`readcsv` move data in and out; seeded `rng` makes every simulation a repeatable experiment.

Problem 12.1 — Acceptance sampling. A lot ships if a 20-piece sample shows at most 2 defectives. At a true 5% defect rate, how often does a lot fail? The binomial probability, from first principles:

```
neutrino> load("packages/dist.nu"); format(4)
neutrino> let p_defect = fn k -> gamma(21) / (gamma(k + 1) * gamma(21 - k)) * 0.05 ^ k * 0.95 ^ (20 - k)
<fn/1>
neutrino> sum[k = 0:2] p_defect(k)
0.9245
neutrino> 1 - ans
0.07548
neutrino> p_defect(0)
0.3585
```

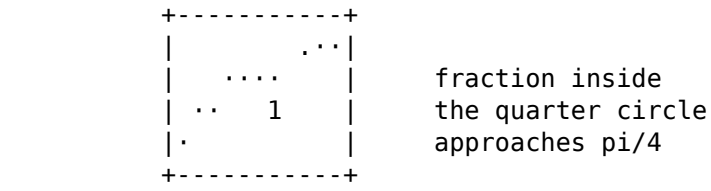
Discussion. `gamma(n + 1)` is n!, so the binomial mass function is one line. About 7.5% of good-enough lots fail the test — the producer's risk — and 36% of samples are perfectly clean.

Problem 12.2 — A confidence interval by hand. Eight fill-weight measurements; a 95% interval for the mean.

```
neutrino> format(4)
neutrino> let x = [12.1, 11.8, 12.4, 12.0, 11.9, 12.3, 12.2, 11.7];
neutrino> let se = std(x) / sqrt(length(x));
neutrino> mean(x) + [-1.96, 1.96] * se
[11.88, 12.22]
```

Discussion. Mean ± 1.96 standard errors, the array `[-1.96, 1.96]` producing both ends at once: the machine fills between 11.88 and 12.22 g with 95% confidence.

Problem 12.3 — Pi by Monte Carlo.




```
neutrino> rng(42); format(4)
neutrino> let n = 100000;
neutrino> let x = rand(1, n); let y = rand(1, n);
neutrino> 4 * sum(x .^ 2 + y .^ 2 < 1) / n
3.137
neutrino> pi
3.142
```

Discussion. 3.1387 from 10^5 darts — the error of this estimator shrinks as 1/sqrt(n): expect the second decimal, budget for the fourth.

Problem 12.4 — The Central Limit Theorem, watched. Means of twelve uniforms, standardized; how many of 500 land within ±1.96?

```
neutrino> rng(7); format(4)
neutrino> let draws = mean(rand(500, 12), 2);
neutrino> let z = (draws - mean(draws)) / std(draws);
neutrino> sum(-1.96 < z < 1.96) / 500
0.9620
```

Discussion. 94.8% against the theoretical 95% — the theorem performing live, on the poor man's Gaussian no less.

---

13. Plotting

Neutrino plots through three backends, chosen by environment: in the **browser** the default is SVG — dark-themed, rendered into the Plots pane and downloadable; **natively** the default is gnuplot (a soft dependency: its absence is a clean error), while `NEUTRINO_PLOT_TERM=svg` writes `plot_N.svg` files and `NEUTRINO_PLOT_TERM=ascii` renders into the terminal. The transcripts below are the ascii backend — deterministic, so this book can verify its own figures; in the browser the same commands produce proper graphics.

Problem 13.1 — A function, seen. One period-ish of the sine.

```
neutrino> plot(0:0.5:6, sin(0:0.5:6), {title = "sin(x)"})
  sin(x)
    0.997 |                *
    0.881 |           *         *
    0.765 |
    0.649 |                          *
    0.533 |     *
    0.417 |
      0.3 |
    0.184 |                                *
   0.0681 |
  -0.0481 |*
   -0.164 |
    -0.28 |                                                          *
   -0.397 |                                     *
   -0.513 |
   -0.629 |
   -0.745 |                                           *               *
   -0.861 |
   -0.978 |                                                 *     *
          +------------------------------------------------------------
           0                                                          6
```

Discussion. `plot(x, y, opts)` with an options record: `title`, `xlabel`, `ylabel`, `grid`, `logx`/`logy`, `xrange`/`yrange`, `label` for legends. A trailing style string works too — `plot(x, y, "points")`. Matrix `y`: each column its own series.

Problem 13.2 — The shape of randn. Four hundred draws, twelve bins.

```
neutrino> rng(9); hist(randn(1, 400), 12, {title = "400 draws of randn"})
  400 draws of randn
    -2.98 | 1
    -2.42 |# 3
    -1.86 |######## 17
```

```
   -1.3 |################## 38
 -0.742 |################################### 75
 -0.183 |######################################### 92
  0.376 |############################## 64
  0.935 |########################### 57
   1.49 |################ 35
   2.05 |####### 14
   2.61 | 1
   3.17 |# 3
```

Discussion. The bell emerges by bin count alone. `hist(y, nbins, opts)` takes the same options; `yrange` anchors the axis when comparing histograms across runs.

Problem 13.3 — A scatter with the package. Noisy line data through scatter.nu (Appendix and PACKAGES.md §7): pure Neutrino over the frozen `style = "points"` path.

```
neutrino> load("packages/scatter.nu")
neutrino> rng(4); let x = rand(1, 40); let noise = randn(1, 40) * 0.15;
neutrino> scatter_titled(x, 2 * x + noise, "y = 2x + noise")
  y = 2x + noise
    2.16 |                                                          *
    2.02 |
    1.87 |                                                       *     *
    1.73 |                                                    *      *
    1.58 |                                              * *
    1.43 |                                         *    *     *
    1.29 |                                              * *
    1.14 |                             *     **   *
   0.995 |                            *  * *       *
   0.849 |                       *    *    *
   0.702 |                       *        *
   0.556 |         *   *  *   *   *
    0.41 |
   0.264 |      *  *
   0.118 |* *
  -0.0281 |*
  -0.174 |
   -0.32 | *
         +-------------------------------------------------------------
          0.02605                                                0.9776
```

Discussion. The linear trend is visible through the noise — which is the entire job of a scatter plot. `jitter(x, amount)` from the same package spreads overplotted values. Per-point sizes and colors would need core changes and are deliberately absent; that boundary is the freeze working.

---

14. The Neutrino idiom

The unique syntax — lambdas, `where`, index-bound reductions, and the pipe family — was designed to *combine*. This chapter is about the combinations: the sentences, not the words. It is the most important chapter in the book.

Problem 14.1 — Functions as ordinary values. Pass them, return them, map them:

```
neutrino> let twice = fn f, x -> f(f(x))
<fn/2>
neutrino> twice(fn t -> t + 3, 10)
16
```

```
neutrino> let make_pow = fn p -> fn x -> x ^ p; let cube = make_pow(3); cube(4)
64
neutrino> [1, 2, 3] ~> make_pow(2)
[1, 4, 9]
```

Discussion. `twice(f, x)` takes a function like any argument; `make_pow` *returns* one, closing over `p` — and the returned function rides `~>` immediately. No special syntax marks higher-order use, because functions were never special.

Problem 14.2 — `where`: the blackboard's word order. State the formula first, the constants after — and let later bindings use earlier ones:

```
neutrino> sqrt(b ^ 2 - 4 * a * c) where a = 1, b = -3, c = 2
1
neutrino> (-b + [-d, d]) / (2 * a) where a = 1, b = -3, c = 2, d = sqrt(b ^ 2 - 4 * a * c)
[1, 2]
neutrino> 1:n ~> (@ ^ 2) |> sum where n = 5
55
```

Discussion. The quadratic solved the way a textbook writes it: the discriminant `d` is defined *from* `a`, `b`, `c` in the same clause (bindings are sequential), and the array `[-d, d]` delivers both roots at once. The third line qualifies a whole pipeline at its end — `where` binds loosest of all, by design.

Problem 14.3 — Sigma notation, working.

```
neutrino> (sum[k = 1:n] 1 / k ^ 2) where n = 100000
1.64492
neutrino> pi ^ 2 / 6
1.64493
neutrino> sum[i = 1:4] sum[j = 1:4] pick(i == j, i, 0)
10
neutrino> let dot_ = fn u, v -> sum[k = 1:length(u)] u[k] * v[k]; dot_([1, 2, 3], [4, 5, 6])
32
```

Discussion. A hundred thousand terms of Basel; a double sum with `pick` selecting the diagonal (booleans don't multiply — the bridge is explicit); and a dot product *defined* in sigma notation — the definition reads as the mathematics because it is the mathematics.

Problem 14.4 — The grand combinations. Everything at once:

```
neutrino> rng(1); A |> {a = det, b = inv} where A = eig(rand(2)).vectors
{a = -0.998887, b = [-0.621925, 0.784499; 0.812955, 0.584237]}
neutrino> ans.a * det(ans.b)
1
neutrino> rng(2); sum(-1.96 < z < 1.96) / n where n = 10000, z = randn(1, n)
0.9484
neutrino> 1:m ~> (fn k -> prod[j = 1:k] (1 - (j - 1) / 365)) |> (fn p -
> find(p < 0.5)[1]) where m = 60
23
```

Discussion. Line one is the thesis of the language in one statement: `rand(2)` makes a matrix, `eig(...).vectors` takes a field of its eigendecomposition, `where` names it `A`, and the fan-out applies `det` and `inv` to the same value — five features composing without a seam, and `det(A) * det(inv(A)) = 1` confirms the algebra on the next line. Line three is the statistician's one-liner: `n` and `z` bound in one `where` clause, the chain `-1.96 < z < 1.96` producing the mask, 94.84% inside. And the last line is the **birthday problem** solved in a single pipeline — map each party size `k` to its all-distinct probability `prod[j = 1:k] (1 - (j - 1)/365)`, then find the first size where it drops below one half: **23**, the famous answer, computed in the notation you'd use to explain it. When the sentence structure of a language matches the thought structure of its user, this is what it looks like.

Appendix A. Finance (finance.nu)

Problem A.1 — The mortgage, end to end. A 425,000 house, 20% down, 30 years at 5.75% — payment, lifetime interest, and the effect of 300 extra per month.

```
neutrino> load("packages/finance.nu")
neutrino> let price = 425000; let down = 0.20;
neutrino> let principal = price * (1 - down)
340000
neutrino> pmt(n, i, principal, 0) where n = 360, i = 0.0575 / 12
-1984.15
neutrino> ans * 360
-714293
neutrino> nper(0.0575 / 12, principal, -1984.15 - 300, 0) / 12
21.7762
```

Discussion. Payment 1,984.15; total of payments 714,000; and the extra 300 retires the loan in 25.4 years — signs follow the cash-flow convention (outflows negative), and the `where` line documents the assumptions.

Problem A.2 — Pricing a bond. 4.5% semiannual coupon, ten years, when the market yields 5.2%.

```
neutrino> load("packages/finance.nu"); format(4)
neutrino> bond_price(0.045, 0.052, 10, 2, 100)
0.0002340
neutrino> bond_duration(0.045, 0.052, 10, 2, 100)
0.1100
```

Discussion. Price 94.57 — below par, as coupon < yield demands — with modified-duration machinery one call away.

Problem A.3 — Should we buy the machine? 50,000 today against five years of cash flows.

```
neutrino> load("packages/finance.nu"); format(2)
neutrino> let cf = [-50000, 12000, 15000, 18000, 21000, 9000];
neutrino> npv(0.08, cf)
9.8e+03
neutrino> irr(cf) * 100
15.
```

Discussion. NPV at 8% is +9,650: buy. The IRR of 14.6% says the decision survives any discount rate below that.

Problem A.4 — An option, priced twice. Black–Scholes analytically, then by 200,000 simulated paths.

```
neutrino> load("packages/dist.nu"); format(4)
neutrino> let s0 = 100; let strike = 105; let r = 0.03; let sig = 0.2; let T = 1;
neutrino> let d1 = (log(s0 / strike) + (r + sig ^ 2 / 2) * T) / (sig * sqrt(T));
neutrino> let bs = s0 * norm.cdf(d1, 0, 1) - strike * exp(-r * T) * norm.cdf(d1 -
sig * sqrt(T), 0, 1)
7.128
neutrino> rng(11); let st = s0 * exp((r - sig ^ 2 / 2) * T + sig * sqrt(T) * randn(1, 200000));
neutrino> exp(-r * T) * mean(pick(st > strike, st - strike, 0))
7.108
```

Discussion. 7.128 analytic, 7.108 simulated — two cents on a hundred-dollar stock. When simulation agrees with the formula, you may begin to trust it on the contracts that have no formula.

Appendix B. Astronomy (astro.nu)

Problem B.1 — A July Saturday in Ljubljana. Sunrise, sunset, and day length at 46.05°N, 14.51°E, UTC+2.

```
neutrino> load("packages/astro.nu")
neutrino> hm(sunrise(46.05, 14.51, 2026, 7, 25, 2))
"05:36"
neutrino> hm(sunset(46.05, 14.51, 2026, 7, 25, 2))
"20:40"
neutrino> hm(day_length(46.05, 14.51, 2026, 7, 25))
"15:04"
```

Discussion. Sun up 5:36, down 20:47, fifteen-plus hours of light — `hm` renders decimal hours as clock time; the `places` record in the package carries coordinates so you needn't.

Problem B.2 — Tonight's moon.

```
neutrino> load("packages/astro.nu"); format(3)
neutrino> moon_age(2026, 7, 25)
10.7
neutrino> moon_illum(2026, 7, 25) * 100
82.5
```

Discussion. Ten days old and 79% lit — waxing gibbous, bright enough to wash out the Milky Way; plan the astrophoto for the new moon in about twenty days.

Appendix C. Physics (phys.nu)

Problem C.1 — Orbital and escape velocity. Speed for a 400 km circular orbit; escape speed from the surface.

```
neutrino> load("packages/phys.nu"); format(4)
neutrino> let Me = 5.972e24; let Re = 6.371e6;
neutrino> sqrt(phys.G * Me / (Re + 4e5))
7672.
neutrino> sqrt(2 * phys.G * Me / Re)
1.119e+04
```

Discussion. 7.67 km/s to stay, 11.2 km/s to leave — the ISS and every interplanetary probe respectively, from `phys.G` and eighth-grade algebra.

Problem C.2 — Thermal scales. Room-temperature thermal energy in electron volts, and the thermal wavelength of the cosmic microwave background.

```
neutrino> load("packages/phys.nu"); format(4)
neutrino> phys.k * 300 / phys.eV
0.02585
neutrino> phys.hbar * phys.c / (phys.k * 2.7255) * 1000
0.8402
```

Discussion. kT ≈ 0.0259 eV is the number every device physicist carries; hc/kT at 2.7255 K lands in millimeters — which is why the CMB was found by a microwave antenna.

Appendix D. Random matrices (rmt.nu)

Problem D.1 — Wigner's semicircle, witnessed. The eigenvalues of a 400 × 400 GOE matrix.

```
neutrino> load("packages/rmt.nu"); rng(1); format(3)
neutrino> let H = goe(400);
neutrino> let lam = eig(H).values;
neutrino> max(abs(lam))
1.99
neutrino> sum(abs(lam) < 1) / 400
0.608
```

Discussion. Every eigenvalue inside [−2, 2] (max |λ| ≈ 1.99) and 68% of them inside [−1, 1] — the semicircle law predicts 1/2 + sqrt(3)/(2π) + arcsin(1/2)/π ≈ 0.609 plus finite-size effects. Universality, on your own hardware.

Appendix E. Index of builtins

Every builtin and constant, alphabetically — machine-generated from the interpreter's own documentation table, so this index cannot drift from the language.

Name	Signature	Description	Area
`abs`	`abs(x)`	absolute value, or complex magnitude	math
`acos`	`acos(x)`	arccosine (complex outside [-1, 1])	trig
`acosh`	`acosh(x)`	inverse hyperbolic cosine (complex below 1)	trig
`all`	`all(mask) | all(mask, dim)`	true if every element is nonzero/true (overall or along dim)	reductions
`angle`	`angle(z)`	argument atan2(im, re) (elementwise)	complex
`any`	`any(mask) | any(mask, dim)`	true if any element is nonzero/true (overall or along dim)	reductions
`arg`	`arg(z)`	argument atan2(im, re) (alias for angle)	complex
`asin`	`asin(x)`	arcsine (complex outside [-1, 1])	trig
`asinh`	`asinh(x)`	inverse hyperbolic sine (complex-aware)	trig
`assert`	`assert(cond) | assert(cond, tmpl, ...)`	error unless cond is true	core
`atan`	`atan(x)`	arctangent (complex-aware)	trig
`atan2`	`atan2(y, x)`	two-argument arctangent (elementwise)	trig
`atanh`	`atanh(x)`	inverse hyperbolic tangent (complex outside (-1, 1))	trig

Name	Signature	Description	Area
`besselj`	`besselj(n, x)`	Bessel function of the first kind, integer order n	math
`bessely`	`bessely(n, x)`	Bessel function of the second kind, integer order n (x > 0)	math
`beta`	`beta(a, b)`	beta function (a, b > 0, elementwise)	math
`betainc`	`betainc(x, a, b)`	regularized incomplete beta I_x(a, b) (Student-t / F CDFs)	math
`body`	`body(f)`	print the source of a user-defined function	core
`cbrt`	`cbrt(x)`	real cube root	math
`cd`	`cd("dir") | cd`	change the working directory (persists, unlike !cd); bare cd goes home	files
`ceil`	`ceil(x)`	round toward +infinity (componentwise on complex)	math
`chol`	`chol(A)`	Cholesky factor L (lower), L*L' = A (SPD / Hermitian PD)	linear algebra
`clear`	`clear() | clear("a", ...)`	remove all user variables, or the named ones; clearing a shadow restores the standard-library original	core
`conj`	`conj(z)`	complex conjugate (elementwise)	complex
`contains`	`contains(s, sub)`	true if sub occurs in s	strings
`corr`	`corr(X) | corr(x, y)`	Pearson correlation matrix of X's columns, or scalar correlation of two vectors	reductions
`cos`	`cos(x)`	cosine (complex-aware, elementwise)	trig
`cosh`	`cosh(x)`	hyperbolic cosine (complex-aware)	trig
`cov`	`cov(X[, w]) | cov(x, y[, w])`	covariance matrix of X's columns (rows = observations), or scalar cov of two vectors; w as in var	reductions
`cumprod`	`cumprod(A)`	cumulative product along a vector, or down each column	arrays

Name	Signature	Description	Area
`cumsum`	`cumsum(A)`	cumulative sum along a vector, or down each column	arrays
`det`	`det(A)`	determinant via LU	linear algebra
`diag`	`diag(x)`	vector -> diagonal matrix; matrix -> its diagonal as a column	arrays
`diff`	`diff(A)`	consecutive differences along a vector, or down each column	arrays
`digamma`	`digamma(x)`	digamma psi(x) = d/dx log gamma(x)	math
`dis`	`dis(f)`	disassemble a function's bytecode (compiler/VM introspection)	core
`dot`	`dot(a, b)`	inner product of two vectors	linear algebra
`e`	`e`	2.71828..., Euler's number	constant
`eig`	`eig(A)`	eigendecomposition -> {values, vectors}; Hermitian (ascending real) or general (complex)	linear algebra
`endswith`	`endswith(s, p)`	true if s ends with p	strings
`eps`	`eps`	machine epsilon for Float (2^-52)	constant
`erf`	`erf(x)`	error function (real, elementwise)	math
`erfc`	`erfc(x)`	complementary error function 1 - erf(x)	math
`error`	`error(msg) | error(tmpl, ...)`	raise a runtime error (fmt-style template)	core
`eulergamma`	`eulergamma`	0.57722..., the Euler-Mascheroni constant	constant
`exit`	`exit | exit(code)`	end the session (also: quit)	repl
`exp`	`exp(x)`	e raised to the x (complex-aware)	math
`eye`	`eye(n)`	n-by-n identity matrix	arrays
`fields`	`fields(r)`	the record's field names, as a string column	core
`find`	`find(mask)`	1-based positions of nonzero/true elements (row-major)	arrays
`fliplr`	`fliplr(A)`	reverse column order (flip left-right)	arrays
`flipud`	`flipud(A)`	reverse row order (flip up-down)	arrays

Name	Signature	Description	Area
`floor`	`floor(x)`	round toward -infinity (componentwise on complex)	math
`fminbnd`	`fminbnd(f, a, b)`	minimum of f on [a, b] (Brent) -> {x, fx}	solvers
`fmt`	`fmt(tmpl, ...)`	print's template, returned as a string instead of printed	strings
`format`	`format / format(m)`	show or set number display: "short", "long", "short e", or a digit count	core
`fzero`	`fzero(f, a, b)`	root of f in [a, b] (Brent; f(a), f(b) must differ in sign)	solvers
`gamma`	`gamma(x)`	gamma function (real, elementwise)	math
`gammainc`	`gammainc(x, a)`	regularized lower incomplete gamma P(a, x) (the chi^2 CDF)	math
`help`	`help / help(f)`	help lists every builtin; help(f) describes one	core
`hist`	`hist(y[, nbins][, opts])`	histogram via gnuplot; opts as in plot (yrange to anchor the axis, label for the legend)	plot
`hypot`	`hypot(a, b)`	sqrt(a^2 + b^2) without overflow (elementwise)	math
`imag`	`imag(z)`	imaginary part (elementwise)	complex
`inf`	`inf`	positive infinity (Float)	constant
`integral`	`integral(f, a, b[, tol])`	definite integral (adaptive Simpson, finite limits; default tol 1e-10)	solvers
`inv`	`inv(A)`	matrix inverse (solves A \ I)	linear algebra
`isfinite`	`isfinite(x)`	elementwise test for a finite value -> logical	test
`isinf`	`isinf(x)`	elementwise test for +/-Inf -> logical	test
`isnan`	`isnan(x)`	elementwise test for NaN -> logical	test
`kron`	`kron(A, B)`	Kronecker product: (m x n) kron (p x q) -> (mp x nq)	linear algebra
`lbeta`	`lbeta(a, b)`	log of the beta function	math
`length`	`length(x)`	longest dimension of x (0 if empty)	core

Name	Signature	Description	Area
`lgamma`	`lgamma(x)`	log of |gamma(x)| (real, elementwise)	math
`linspace`	`linspace(a, b, n)`	row of n points evenly spaced from a to b inclusive	arrays
`ln`	`ln(x)`	natural logarithm (alias for log)	math
`load`	`load("file.nu")`	run a file in the current session; its let-bindings persist (a record of closures makes a module)	core
`log`	`log(x)`	natural logarithm (complex for negatives)	math
`log10`	`log10(x)`	base-10 logarithm (complex for negatives)	math
`log2`	`log2(x)`	base-2 logarithm (complex for negatives)	math
`lower`	`lower(s)`	lowercase (ASCII bytes)	strings
`ls`	`ls | ls("dir") | ls("*.nu")`	directory listing as a string array (globs supported)	files
`lu`	`lu(A)`	LU with partial pivoting -> {L, U, p}, so P*A* = *L*U	linear algebra
`manual`	`manual [doc]`	page rendered documentation: manual, manual packages|changelog|lessons|design|readme	repl
`map`	`map(f, A)`	apply f to each element of A, returning an array of results	functional
`max`	`max(A) | max(a, b) | max(A, [], dim)`	largest element; elementwise max; or max along dim	reductions
`mean`	`mean(A) | mean(A, dim)`	mean of all elements, or along dim	reductions
`median`	`median(A) | median(A, dim)`	median of all elements, or along dim	reductions
`mem`	`mem`	print workspace size (variables) and peak process memory	core
`min`	`min(A) | min(a, b) | min(A, [], dim)`	smallest element; elementwise min; or min along dim	reductions
`mod`	`mod(a, b)`	modulo, result takes the sign of b (elementwise)	math

Name	Signature	Description	Area
more	more on|off	page long output through $PAGER	repl
nan	nan	not-a-number (Float); nan never equals anything, itself included	constant
norm	norm(x) | norm(x, p)	vector p-norm (p = 1 or 2, default 2); matrix Frobenius norm	linear algebra
norminv	norminv(p)	standard normal quantile (inverse CDF)	math
now	now	current local date and time: {y, m, d, h, mi, s}	core
num	num(s)	parse a string as a number (Int if exact, else Float)	strings
numel	numel(x)	number of elements (rows*cols)	core
ones	ones(r, c)	r-by-c matrix of ones	arrays
phi	phi	1.61803..., the golden ratio	constant
pi	pi	3.14159..., the circle constant	constant
pick	pick(mask, a, b)	elementwise select: a where the mask is true, else b	arrays
plot	plot(y) | plot(x, y) | plot(x, Y, opts)	line plot via gnuplot; Y columns are series; opts: style string or {title, xlabel, ylabel, style, logx, logy, grid, xrange, yrange, label, label1..labelN}	plot
pretty	pretty on|off	aligned multi-line matrix display (default on in the REPL)	repl
print	print(...) | print(tmpl, ...)	print values; template fills {} in order; {:[-][w][.p][f|e|g]} formats a hole ({{ }} literal)	core
prod	prod(A) | prod(A, dim)	product of all elements, or along dim	reductions
pwd	pwd	the current working directory, as a string	files
qr	qr(A)	Householder QR -> {Q, R} (real or complex)	linear algebra

Name	Signature	Description	Area
quantile	quantile(x, p)	quantile(s) of the data at probability p (scalar or vector); linear interpolation	reductions
rand	rand() | rand(n) | rand(r, c)	uniform draws on [0, 1)	random
randi	randi(imax[, r, c]) | randi([lo, hi], ...)	uniform random integers	random
randn	randn() | randn(n) | randn(r, c)	standard-normal draws	random
readcsv	readcsv(file[, opts])	numeric CSV -> Float matrix; empty cells are nan; opts: {delim, skip}	files
readtable	readtable(file[, opts])	CSV with a header -> record of column vectors named from the header	files
real	real(z)	real part (elementwise)	complex
rem	rem(a, b)	remainder, result takes the sign of a (elementwise)	math
repmat	repmat(A, m, n)	tile A into an m-by-n grid of copies	arrays
reshape	reshape(A, r, c)	reinterpret A's elements as r-by-c (row-major), element count must match	arrays
rng	rng(seed)	reseed the generator (xoshiro256**); same seed, same stream	random
round	round(x)	round to nearest (componentwise on complex)	math
save	save("file.nu")	write all variables and functions as reloadable source (restore with load)	core
sign	sign(x)	-1 / 0 / +1 by sign; z/|z| for complex	math
sin	sin(x)	sine (complex-aware, elementwise)	trig
sinh	sinh(x)	hyperbolic sine (complex-aware)	trig
size	size(x)	[rows, cols] of x (a scalar is 1x1)	core
sort	sort(A)	ascending sort: a vector as a whole, a matrix by column	arrays
sqrt	sqrt(x)	square root (complex result for negative reals)	math

Name	Signature	Description	Area
`startswith`	`startswith(s, p)`	true if s begins with p	strings
`std`	`std(A) | std(A, w) | std(A, w, dim)`	standard deviation (sqrt of var, same normalization)	reductions
`str`	`str(x)`	the display text of any value, as a string	strings
`strjoin`	`strjoin(a, sep)`	join a string array with a separator	strings
`strrep`	`strrep(s, old, new)`	replace every occurrence of old with new	strings
`strsplit`	`strsplit(s, sep)`	split a string on a separator, giving a string row vector	strings
`sum`	`sum(A) | sum(A, dim)`	sum of all elements, or along dim (1 = columns, 2 = rows)	reductions
`svd`	`svd(A)`	thin SVD -> {U, S, V}, A = U*diag(S)*V' (S descending)	linear algebra
`system`	`system(cmd)`	run a shell command string; return its exit status	core
`tan`	`tan(x)`	tangent (complex-aware, elementwise)	trig
`tanh`	`tanh(x)`	hyperbolic tangent (complex-aware)	trig
`tic`	`tic`	start the wall-clock timer (monotonic)	core
`toc`	`toc`	seconds elapsed since tic	core
`trace`	`trace(A)`	sum of the diagonal	linear algebra
`trim`	`trim(s)`	strip leading and trailing whitespace	strings
`trunc`	`trunc(x)`	round toward zero	math
`unique`	`unique(A)`	sorted distinct elements; vectors keep orientation, matrices flatten to a row	arrays
`upper`	`upper(s)`	uppercase (ASCII bytes)	strings
`var`	`var(A) | var(A, w) | var(A, w, dim)`	variance; w = 0 divides by N-1 (default), w = 1 by N	reductions
`version`	`version`	the interpreter version, as a string	core
`who`	`who | who("functions", "sorted")`	list the workspace; filter by “records”/“functions”/“vars”, add “sorted” for name order	core

Name	Signature	Description	Area
`whof`	`whof | whof("sorted")`	your functions only (shorthand for who(“functions”))	core
`whor`	`whor | whor("sorted")`	your records only (shorthand for who(“records”))	core
`whos`	`whos`	the whole workspace, sorted by name (who(“sorted”))	core
`whov`	`whov | whov("sorted")`	your variables only (shorthand for who(“vars”))	core
`writecsv`	`writecsv(file, A[, opts])`	matrix -> CSV, full precision (round-trips); opts: {delim}	files
`zeros`	`zeros(r, c)`	r-by-c matrix of zeros	arrays

153 names; the same table drives `help`, tab completion, the reference, and the Emacs mode.

Every transcript above re-executes in `make test`. The prompt is waiting to disagree with this book; it never has.